

Test Coverage & Feature-Gap Report — IISc Grounded Agentic RAG Platform

- **Date:** 2026-06-21
- **Branch evaluated:** `codex/evaluation` (contains all of `dev` ; `dev` is 0 commits ahead)
- **Companion document:** `END_TO_END_EVALUATION_REPORT_20260621T2029.md` (live RAGAS results + prioritized fixes)
- **Purpose:** (a) what *additional* test coverage the solution needs, and (b) for each shipped feature, the functionality that current test cases do **not** exercise — with concrete test cases to add.

This report answers Step 4 of the evaluation brief: "what else should be covered as tests/features, and where individual features have functionality not covered by the current test cases, suggest them."

1. Current Automated Coverage (baseline)

Layer	What exists	Count
Backend unit/integration (<code>tests/</code>)	12 files, async pytest, auto-skip if no Postgres	~138 test functions
Frontend component (<code>frontend/src/components/chat/*.test.tsx</code>)	Vitest + RTL, chat components only	5 files
Evaluation harness (<code>evaluation/</code>)	RAGAS suite + dataset validation + reporting	148-case suite, 13 apps
CI (<code>.github/workflows/ci.yml</code>)	backend pytest (MOCK_N8N), backend docker build, frontend test+build	3 jobs

Well-covered modules: M2 (auth), M3 (admin docs/conversations + units), M4 (chat, slack, titles), M5 (ingestion chunking), M12 (WhatsApp), M14 (Teams), M10 (eval harness).

Coverage by module (condensed):

Module	Area	Tests	Verdict
M2	Auth, JWT, rate-limit	<code>test_auth.py</code> (12)	Good
M3	Admin docs, conversations, units	72 across 3 files	Strong
M4	Chat, Slack, onboarding, title	27 across 4 files	Good
M5	Ingestion chunking/embedding	<code>test_ingestion_pipeline.py</code> (21)	Good (embedding mocked)
M6	Retrieval pipeline	0 (live <code>run_eval</code> only)	Gap
M7	DB schema / migrations	implicit only	Weak
M8	Admin UI	0	Gap
M9	Chat UI	5 components	Partial
M10	Eval harness	<code>test_evaluation_dataset.py</code> (10)	Good
M11	Docs / MCP server	0 for MCP	Gap
M12	WhatsApp	<code>test_m12_whatsapp.py</code> (10)	Strong
M13	Onboarding (API + UI)	API via slack-onboard (6); UI 0	Partial
M14	Teams	<code>test_m14_teams.py</code> (12)	Strong

2. Highest-Priority Coverage Gaps (add these first)

These are ordered by risk. Each maps to a concrete, automatable test.

G1 — Multi-tenant isolation is asserted but never tested end-to-end (CRITICAL)

The platform's core security promise is tenant isolation, yet no test proves that tenant A cannot read tenant B's data through any path. The live retrieval pipeline's tenant filter is a recent, hand-applied fix (see companion report) — exactly the kind of regression a test must guard.

Add:

- Seed two tenants with distinct documents; assert `/chat/query` for tenant A never returns tenant B's chunks (sources + answer text).
- Assert `/admin/documents` (list/get/delete/download) is 404/403 across tenants.
- Assert conversation GET/DELETE rejects cross-tenant `conversation_id` (403).
- **n8n contract test:** POST `/webhook/retrieve` with tenant A's `tenant_id` and a query whose answer only exists in tenant B's corpus → expect abstention / no B sources.

G2 — M6 retrieval pipeline has no automated test at any level (CRITICAL)

The whole RAG quality depends on `retrieval-pipeline.json`, which is only ever exercised by the manual `run_eval`. There is no guard that the webhook returns `sources`, scopes by tenant, reads `current_message`, or uses a valid model.

Add a contract/smoke test (CI-runnable against a stub or the live webhook behind a flag):

- Response schema test: `{answer:str, sources:[{chunk_text,title}], model_used}` present.
- "Returns ≥1 source for a known-answerable question."
- "Reads `current_message`" — send only `current_message` (no `query`) and assert a non-empty, grounded answer (this is the exact bug that shipped).
- "Tenant filter" as in G1.
- A pinned **golden-answer** regression test for 3–5 canonical questions (e.g. S1 ack = "15 minutes") to catch silent quality regressions.

G3 — Negative / abstention behavior is only measured in aggregate (HIGH)

`run_eval` computes a `negative_abstention_rate`, but there is no per-case unit test that a specific unanswerable question yields a refusal rather than a hallucination. Abstention is a primary correctness property of a *grounded* RAG system.

Add: parametrized tests over the 13 suite negatives asserting the answer contains a refusal marker and returns no fabricated citation.

G4 — Admin UI (M8) and Onboarding UI (M13) have zero tests (HIGH)

All admin pages (`/admin/users`, `/admin/tenants`, `/admin/settings`, document detail) and the onboarding flow are untested; several are still mock/placeholder screens. No test catches a regression in document upload, status polling, or the auth guard redirect.

Add (Vitest + RTL, mock `ApiClient`):

- Document list: loading → rows → empty → error states; status badge mapping.
- Upload: file picked → POST called → optimistic row → success/failure toast.
- `AuthGuard`: unauthenticated → redirect to `/login?next=...`; wrong role → blocked.
- Onboarding: slug-availability check, multi-step form validation, submit → tenant created.

G5 — MCP server (`backend/app/mcp/`) is completely untested (HIGH)

If `MCP_ENABLED=true`, `/mcp/info` and `/mcp/rpc` are live, externally reachable, and auth-optional. No test covers tool discovery, JSON-RPC dispatch, API-key enforcement, or tenant scoping of MCP queries.

Add: `/mcp/info` catalog shape; `/mcp/rpc` happy path + invalid method (-32601);
API-key required when configured; MCP query is tenant-scoped like `/chat/query`.

G6 — Database migrations (M7) are never tested (MEDIUM)

`init.sql` + Alembic exist, but nothing asserts `alembic upgrade head` succeeds on a clean DB, that the ORM models match the migrated schema, or that downgrade is safe.

Add a CI job: spin Postgres+pgvector, run `alembic upgrade head` then `downgrade base`, and a `check-migrations` test that autogenerate produces no diff against models.

3. Per-Feature Functionality Not Covered by Existing Tests

For each shipped feature, the specific behaviors current tests miss, and the cases to add.

Auth (M2) — covered: *login, register, /me, rate-limit*

Missing:

- Expired-token rejection (forge a token with past `exp`).
- Tampered-signature / wrong- `alg` (e.g. `none`) rejection.
- Role escalation: a `user` token hitting admin-only routes → 403.
- `super_admin` cross-tenant access is intentional — assert it, so it can't silently break.
- Rate-limit applies per-IP and `resets` after the window (only the 429 burst is tested).
- Password hashing: bcrypt verify round-trip + rejection of a wrong password at the `security.py` level (currently only exercised indirectly).

Document upload / ingestion (M3/M5) — covered: *MIME+magic, R2/local, n8n callback*

Missing:

- **Upload-security adversarial cases:** zip-bomb, polyglot (valid PDF header + script payload), oversize-by-1-byte (boundary), zero-byte file, filename path traversal (`../..etc`), double extension (`x.pdf.exe`).
- **Quota edges:** the 20-uploads/hour and storage-cap boundaries (at limit, over limit → 413), and that the rolling window in `upload_audit` actually expires.
- Ingestion failure callback (`status=failed`) updates the document row + surfaces `error_message`; callback with a bad/missing `N8N_CALLBACK_TOKEN` → 401/403.
- Idempotent re-ingestion of the same document (no duplicate chunks).
- Delete cleans up storage and `document_chunks` (cascade) — assert chunk rows gone.

Chat / retrieval flow (M4/M6) — covered: *happy path, foreign-conv 403, history, title*

Missing:

- Multi-turn grounding: a follow-up that depends on prior turns uses the 10-message history correctly (and that history is truncated at 10).
- Reset commands (`/new` , `/reset` , `/clear`) actually recreate the conversation.
- `PipelineError` → 502 + fallback message (path exists; assert the user-facing contract).
- `max_chunks` is honored end-to-end (today the API accepts 1–20 but the pipeline used a fixed LIMIT — assert or fix the mismatch).
- `requires_clarification` / `faithfulness` propagate from pipeline → API → DB (the schema has the columns; the flow does not yet populate them).
- Empty/whitespace query and very long query handling.

Bots — Slack (M4), WhatsApp (M12), Teams (M14) — covered: *sig/token, dedup, posting, maps*

Missing (common to all three):

- **Identity auto-provisioning side effects:** WhatsApp/Teams auto-create users with synthetic emails (`*.whatsapp.local` , `*.teams.local`). No test covers the unmapped-tenant path, duplicate-user prevention on rapid messages, or that an unknown sender is rejected.
- **Reply truncation/formatting:** WhatsApp 1600-char cap, source list capped at 3, Block Kit / Markdown rendering — assert formatting is stable.
- **Replay/idempotency under concurrency:** two identical webhook deliveries within the dedup

window (Slack `processed_requests`) — only the single-delivery path is tested.

- **Ephemeral upload retrieval (the actual user value):** a media upload is indexed into `ephemeral_chunks` AND a subsequent question retrieves it. Today retrieval only searches `document_chunks` for the main path — see companion report — so this likely *fails*; a test would prove it.
- Signature/token **negative** cases beyond the basics (clock skew on Teams JWT `exp`, wrong audience/issuer, Twilio signature with tampered body).

Conversations (M3) — covered: *list, get, delete, ownership*

Missing: pagination/ordering of the conversation list; soft-delete semantics (deleted conversations excluded from list but messages retained per schema); deleting a conversation mid-stream.

Frontend chat (M9) — covered: *5 components*

Missing: `MessageList` autoscroll + streaming/typing transitions; `CitationsPanel` with malformed/empty sources; **the incomplete 401 handler** (`ApiClient` catches 401 but does not clear the token / redirect — add a test that 401 logs the user out); conversation-detail page actually fetching messages (currently a placeholder).

Onboarding (M13) — covered: *Slack user-linking only*

Missing: the self-service `/onboarding/register` and `/onboarding/check-slug` are `501` stubs — when implemented, need slug uniqueness/validation, tenant+admin creation atomicity, and duplicate-email handling.

4. Test-Infrastructure Improvements

- **Run integration tests in CI for real.** Tests auto-skip when Postgres is unreachable; CI must guarantee a pgvector service so the ~138 tests actually execute (today a skip looks green). Add a coverage gate and fail CI if the skip count is non-zero.
- **A typed webhook contract.** The FastAPI↔n8n boundary is untyped JSON and has already produced four shipped bugs (query field, tenant filter, sources, model id). Define Pydantic request/response models for every n8n webhook and add contract tests on both sides.
- **Make `run_eval` a gated CI job (nightly).** Run the suite against the live endpoint with `--enforce-thresholds`; publish the HTML bundle as an artifact and alert on regression.
- **Golden/regression set.** Freeze ~10 canonical Q→A pairs as exact-match-ish regression tests so quality drops are caught without a full RAGAS run.
- **E2E smoke (Playwright).** One browser path: login → upload doc → wait for "completed" → ask a question → see a grounded answer with a citation. There is currently no E2E test.
- **Load/latency test.** p95 latency is ~20s+ (see companion report); add a small concurrency test asserting the endpoint stays correct (no cross-talk, no 5xx) under N parallel queries.

5. Feature Gaps to Cover Next (beyond tests)

Features that are specified or expected but not yet shipped / complete — each should land *with* the tests in §2–§3.

#	Feature	State today	Add
F1	Ephemeral-upload Q&A (WhatsApp/Teams/chat file → ask about it)	Upload indexes to <code>ephemeral_chunks</code> ; main retrieval doesn't search it	Make retrieval search <code>document_chunks</code> u <code>ephemeral_chunks</code> (conversation-scoped); test G-style
F2	Self-service onboarding (<code>/onboarding/*</code>)	<code>501</code> stubs; no UI	Implement tenant+admin+first-doc wizard; slug check
F3	Admin user management / invites	<code>/admin/users/invite</code> 501; UI placeholder	Invite-by-email + role assignment + UI
F4	Tenant management (super_admin)	List/create 501; UI placeholder	CRUD + UI
F5	Grounding signals surfaced (<code>faithfulness</code> , <code>requires_clarification</code> , follow-ups)	Schema + UI badges exist; pipeline doesn't return them	Emit from n8n self-check; populate DB; show in UI
F6	Observability	stdout logs, <code>/health</code> only	Structured JSON logs, request timing, error tracking (Sentry), n8n+frontend health checks
F7	Token revocation / refresh	Stateless JWT, no logout invalidation	Token-version column or short-lived access + refresh
F8	Per-endpoint rate limiting	Only <code>/auth/login</code>	Limit <code>/chat/query</code> and <code>/admin/documents/upload</code>
F9	Document detail + chunk inspection UI	Mock data	Wire to API; show <code>chunk_count</code> , <code>page_count</code> , errors
F10	Cost/usage tracking	Referenced in docs; script absent	Implement <code>scripts.cost_report</code> ; per-tenant token accounting

6. Quick-Win Checklist (cheap, high-value)

- [] Cross-tenant isolation test (G1) — blocks the platform's #1 risk.
- [] M6 webhook contract + golden-answer test (G2).
- [] Parametrized abstention tests over the 13 negatives (G3).
- [] `AuthGuard` redirect + 401-logout frontend tests (G4, §3 frontend).
- [] CI: guarantee Postgres so integration tests run; fail on skips.
- [] Adversarial upload-security tests (§3 ingestion).
- [] MCP `/mcp/rpc` auth + dispatch tests (G5).
- [] Alembic upgrade/downgrade CI job (G6).